Distributed Task Execution System

Master–Worker Architecture

System Problem

Desired System Behavior:

- Multiple laptops sharing work equally
- Balanced load across all machines
- No single point of failure
- Automatic task distribution

Ideally, all machines in a network should share work dynamically. Tasks should go to the least loaded system, and the system should continue even if one node fails.

Drawbacks previously:

❌ No Decentralized Architecture

- Single point of failure (master crash = system down)
- Poor scalability & bottleneck at central node

❌ No UDP Gossip (Load Sharing)

- No real-time load info
- Wrong worker selection → slower execution

❌ No Fault Tolerance

- Task lost if node crashes
- No recovery or retry

❌ No Timeout Mechanism

- Infinite loops freeze system
- Resource blocking

❌ No Proper Network Config (Firewall)

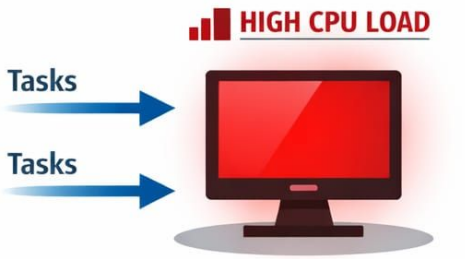
- Nodes can’t connect
- System fails to operate

Objectives

- Eliminate single point of failure (fully decentralized)
- Dynamically distribute tasks based on system load
- Use UDP gossip protocol for load sharing
- Use TCP for reliable execution + result transfer
- Implement fault tolerance (node crash recovery)
- Prevent system freeze using timeout (infinite loop protection)
- Ensure real-time execution (no unnecessary delays)
- Also disabling firewall to ensure any device can participate

Problems in Task Execution Systems

Lack of Distributed Computing

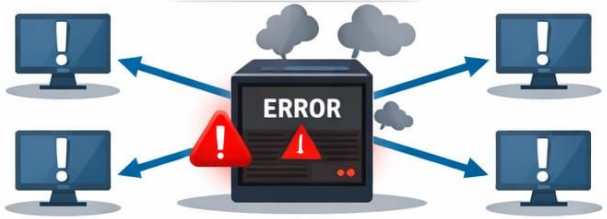


Overloaded Single System

Centralized Bottleneck

Overloaded Central Server

Server Failure



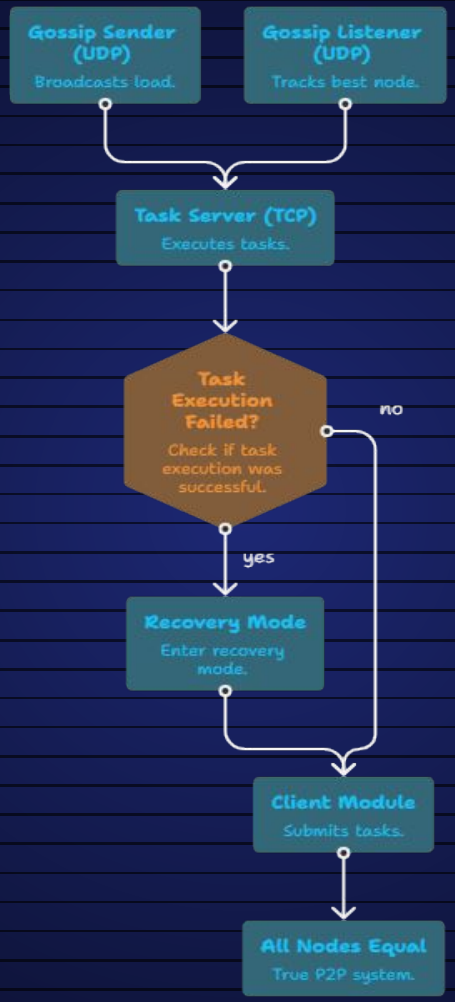
Network Down!

PROPOSED SOLUTION

“Decentralized P2P Task Execution System”

- Fully decentralized architecture
- Every node = client + server
- Dynamic load-based execution

Architecture



KEY FEATURES

1. Gossip-based load sharing (UDP)
2. Load-aware scheduling
3. Fault tolerance
4. Infinite loop protection
5. Real-time execution

SYSTEM ARCHITECTURE

Node A ↔ Node B ↔ Node C ↔ Node D

3 components:

1. Gossip Sender
2. Gossip Listener
3. Task Server

Each node runs three parallel services: one to send load, one to receive load, and one to execute tasks.

TECHNOLOGIES USED

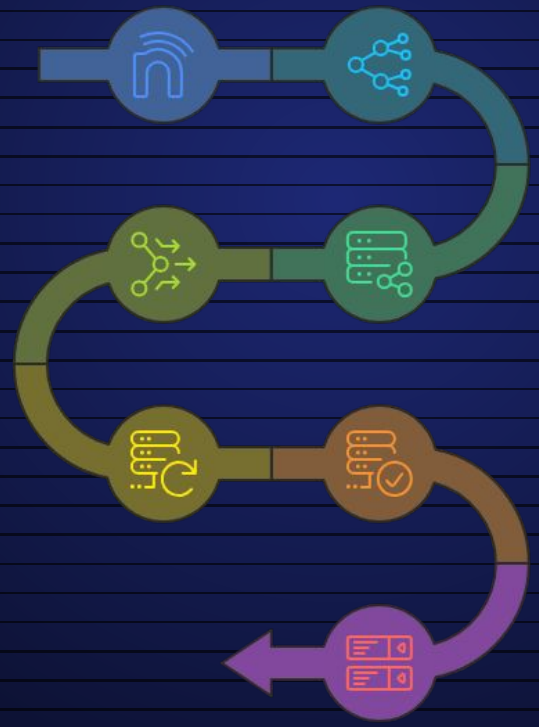
Component to Technology Mapping

Component	Discovery	Execution	Transfer	Threads	Load	CPU	Timeout
Technology	UDP	TCP	SCP	pthreads	getloadavg	get_nprocs	Linux timeout

FAILURE HANDLING

- Crash detection
- Retry mechanism
- Fallback to local execution
- Timeout for infinite loops

EXECUTION FLOW



Node starts

Initial node activation

Gossip exchange (LOAD)

Nodes share load information

Best node selection

Node with lowest load chosen

Task transfer (SCP)

Task sent to best node

Execution (TCP)

Task executed by best node

Recovery Mode

Node enters recovery if task fails

Result return

Results sent back to client

CONCLUSION

1. Efficient resource utilization
2. Scalable system
3. Reliable execution
4. We tested multiple scenarios including crashes and infinite loops, and the system handled all cases correctly.

Thanks!